Centro Federal de Educação Tecnológica de Minas Gerais
Professora: Poliana Corrêa

RELATÓRIO PRÁTICA 7 - LAOC
UNIDADES DE CONTROLE PRINCIPAL E CONTROLE DA ULA

ALUNOS:
GIOVANNI ESTEVAM DOS SANTOS
JOÃO VITOR LOPES FAGUNDES

Belo Horizonte - MG
2025

1. Introdução

A Prática 7 (Unidades de Controle Principal e Controle da ULA) , teve como foco principal implementar os dois módulos responsáveis por "comandar" o caminho de dados: a Unidade de Controle Principal e o Controle da ULA . O objetivo é gerar os diversos sinais de controle que determinam o comportamento dos componentes (como multiplexadores, ULA, banco de registradores e memórias) com base na instrução que está sendo executada . Para o seu funcionamento, a Unidade de Controle Principal utiliza o opcode da instrução (Instruction[31-26]) para definir os sinais de alto nível, como ALUSrc , RegWrite , MemRead , MemWrite , Branch e o sinal ALUOp . O Controle da ULA, por sua vez, recebe o sinal ALUOp (vindo do controle principal) e o campo function da instrução (Instruction[5-0]) . Com base nessas entradas, ele gera o código específico que instrui a ULA sobre qual operação exata (como add, sub, AND, OR) ela deve executar .

2. Desenvolvimento

Nosso projeto da Prática 7 consiste na implementação das **Unidades de Controle do processador MIPS** , componentes digitais que implementam a lógica de "comando" do *datapath*. O projeto foi dividido em dois módulos principais, conforme solicitado: a **Unidade de Controle Principal** e o **Controle da ULA**. A arquitetura foca em gerar os sinais de controle necessários para orquestrar os componentes do *datapath* (como multiplexadores, banco de registradores e a própria ULA). Conforme especificado na prática, a Unidade de Controle Principal recebe o *opcode* da instrução para gerar sinais como RegWrite, ALUSrc e Branch , enquanto o Controle da ULA usa o campo *function* e o sinal ALUOp para determinar a operação aritmética/lógica exata.

Descrição e Decisões de Projeto dos Módulos

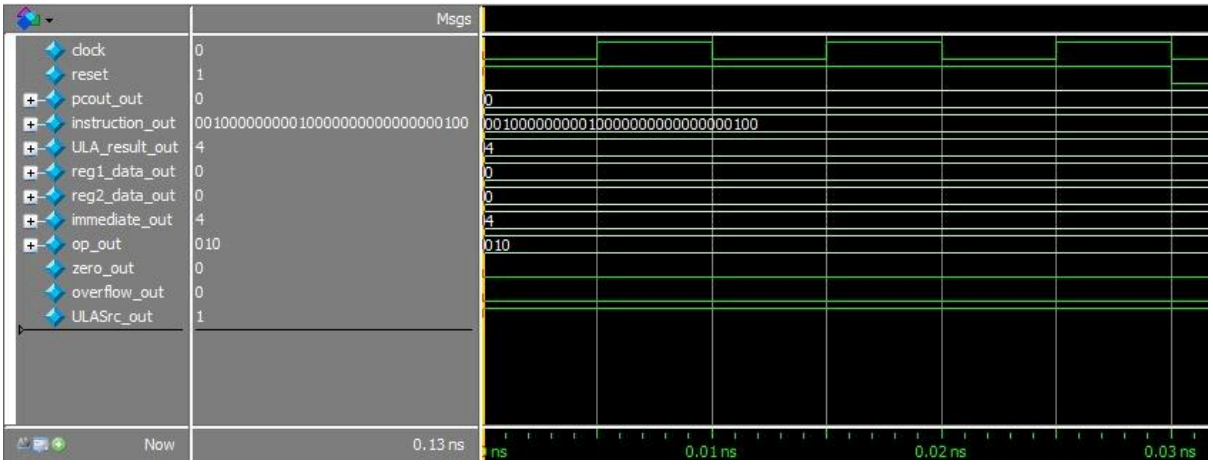
Unidade de Controle Principal: unidadeControlePrincipal

- **Descrição:** O módulo unidadeControlePrincipal implementa a lógica de controle de alto nível do processador. Ele recebe como entrada o *opcode* de 6 bits da instrução e gera os nove sinais de controle que ditam o comportamento dos demais blocos , incluindo RegDst, Branch , MemRead, MemtoReg, MemWrite, ULASrc , RegWrite e os dois bits ULAOp.
- **Decisões de Projeto:**
    - **Implementação Combinacional:** A principal decisão foi implementar o módulo de forma puramente combinacional, usando apenas atribuições contínuas (assign). Isso garante que qualquer mudança no *opcode* de entrada se propague imediatamente para as saídas de controle.
    - **Decodificação de Opcode:** Em vez de uma estrutura case ou always, optamos por decodificar explicitamente os *opcodes* das instruções suportadas (formato-R, lw, sw, beq e addi) em *wires* intermediários. Cada *wire* (rFormat, lw, etc.) é ativado verificando a combinação exata dos 6 bits do opcode.
    - **Geração de Sinais de Controle:** Os sinais de saída são gerados por lógica booleana simples, baseada nos *wires* decodificados, espelhando o diagrama lógico da prática. Por exemplo, o sinal ULASrc é ativado (1) se a instrução for lw, sw OU addi (assign ULASrc = lw | sw | addi;) , pois todas elas usam o extensor de sinal. Da mesma forma, ULAOp[1] é ativado apenas para rFormat e ULAOp[0] apenas para beq , implementando a lógica da Tabela 1.

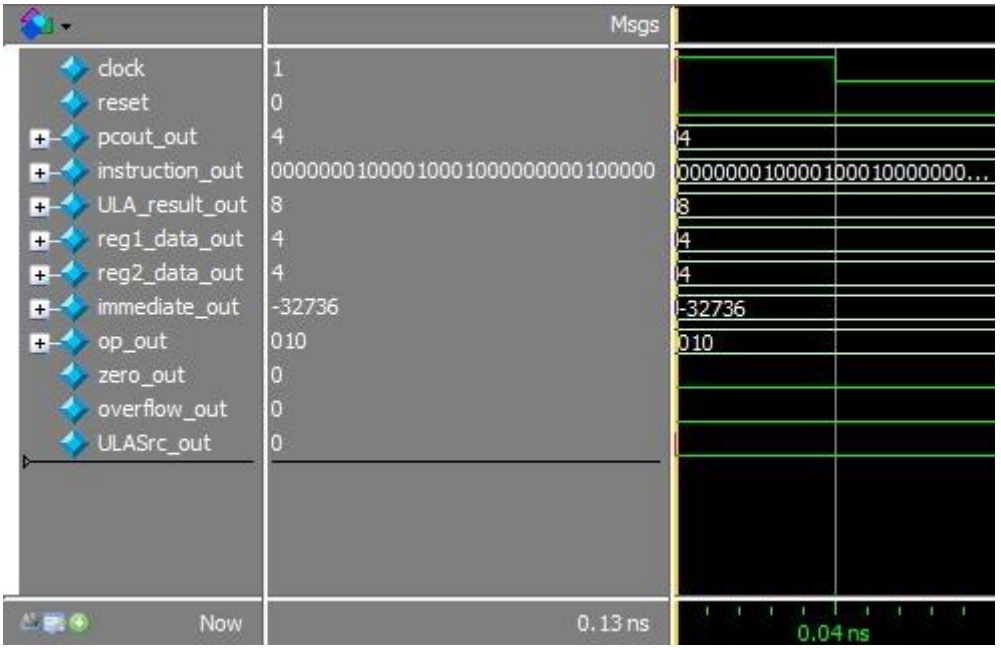
Controle da ULA: unidadeControleULA

- **Descrição:** Este módulo implementa o controle de baixo nível, específico da ULA. Ele recebe o campo funct de 6 bits da instrução e o sinal ULAOp[1:0] (vindo da Unidade de Controle Principal). Com base nessas entradas, ele gera o sinal op[2:0] que seleciona a operação exata (como add, sub, AND, OR) que a ULA deve executar.
- **Decisões de Projeto:**
    - **Lógica Minimizada:** A decisão de projeto foi implementar a tabela verdade (Tabela 1 da prática) de forma minimizada, diretamente em Verilog. Optou-se por atribuições contínuas (assign) para cada bit da saída op em vez de um bloco always com case.
    - **Mapeamento da Tabela Verdade:** A lógica implementada é o resultado direto da minimização da Tabela 1 e do diagrama lógico. Por exemplo, a expressão assign op[2] = (ULAOp[0] | (ULAOp[1] & funct[1])); demonstra essa lógica: ULAOp[1] (ativo para formato-R) habilita a decodificação de funct[1] (para sub ou slt), enquanto ULAOp[0] (ativo para beq) força a operação op[2] para 1 (parte da operação de subtração).

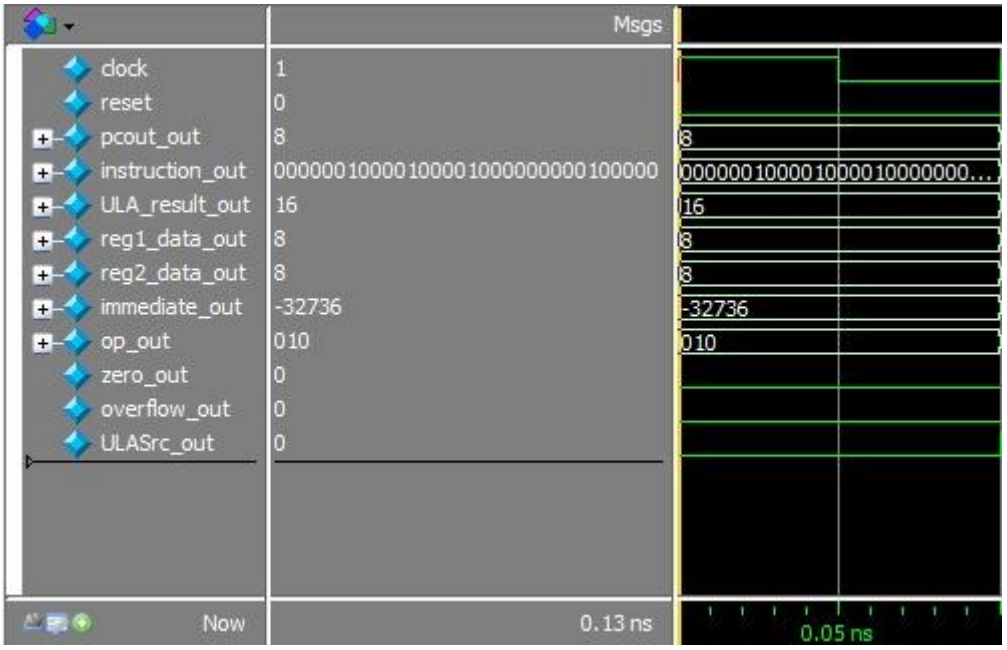
3. Simulações



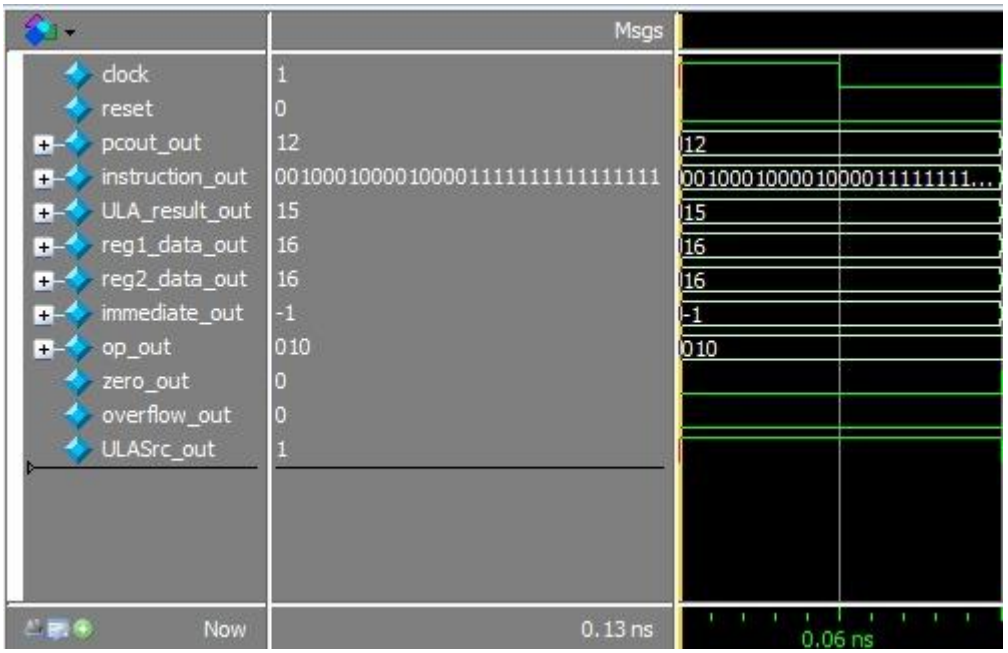
No tempo inicial, o contador pc está em 0 e o reset está em 1, mantendo o processador em estado de reset. O reg 1 ($zero) é 0 e o imediato é 4, a operação que será realizada será um addi $t0, $zero, 4 => $t0 = 4. ULASrc_out está setada em 1, o que evidencia que o valor selecionado no MUX anterior à ULA veio do imediato.



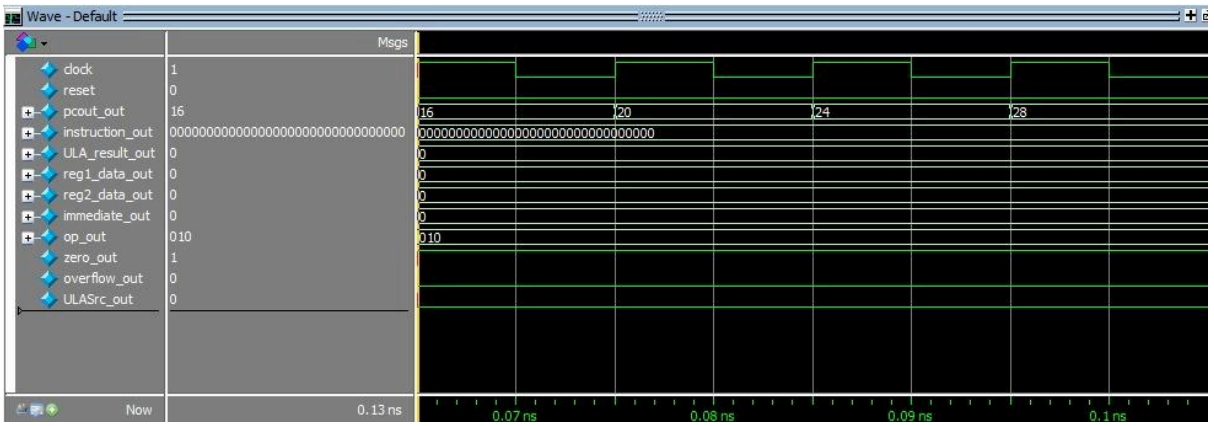
No primeiro ciclo de clock, depois de soltar o reset, PC avança para a próxima instrução e vai para PC + 4 = 4. Os registradores 1 e 2 são o $t0, que possui o valor 4 no momento, mostrado em "reg1_data_out" e "reg2_data_out". A operação que será realizada é um add $s0, $t0, $t0 => $s0 = 4 + 4 = 8, tal resultado é mostrado em ULA_result_out, que contem 8. Nesse clock, o ULASrc_out = 0 mostra que o segundo operando veio do registrador, não do imediato.



No segundo clock, PC avança para a próxima instrução: PC + 4 = 8. Os reg 1 e 2 agora estão mostrando o valor contido no registrador $s0 (8 após a soma no ciclo anterior). A operação realizada é um add $s0, $s0, $s0, => $s0 = 8 + 8 = 16, tal resultado é mostrado no registrador ULA_result_out.



No terceiro clock, PC avança para a próxima instrução: PC + 4 = 12. O reg 1 mostra o valor contido no registrador $s0 (16 após a soma no ciclo anterior). A operação realizada é um addi $s0, $s0, -1, => $s0 = 16 - 1 = 15, tal resultado é mostrado no registrador ULA_result_out. Como é uma operação do tipo I, o ULASrc_out = 1, mais uma vez, pega, como segundo operando da ULA, o valor que vem do extensor de sinal.



Depois disso, todas as instruções são zeradas e o PC segue contando.